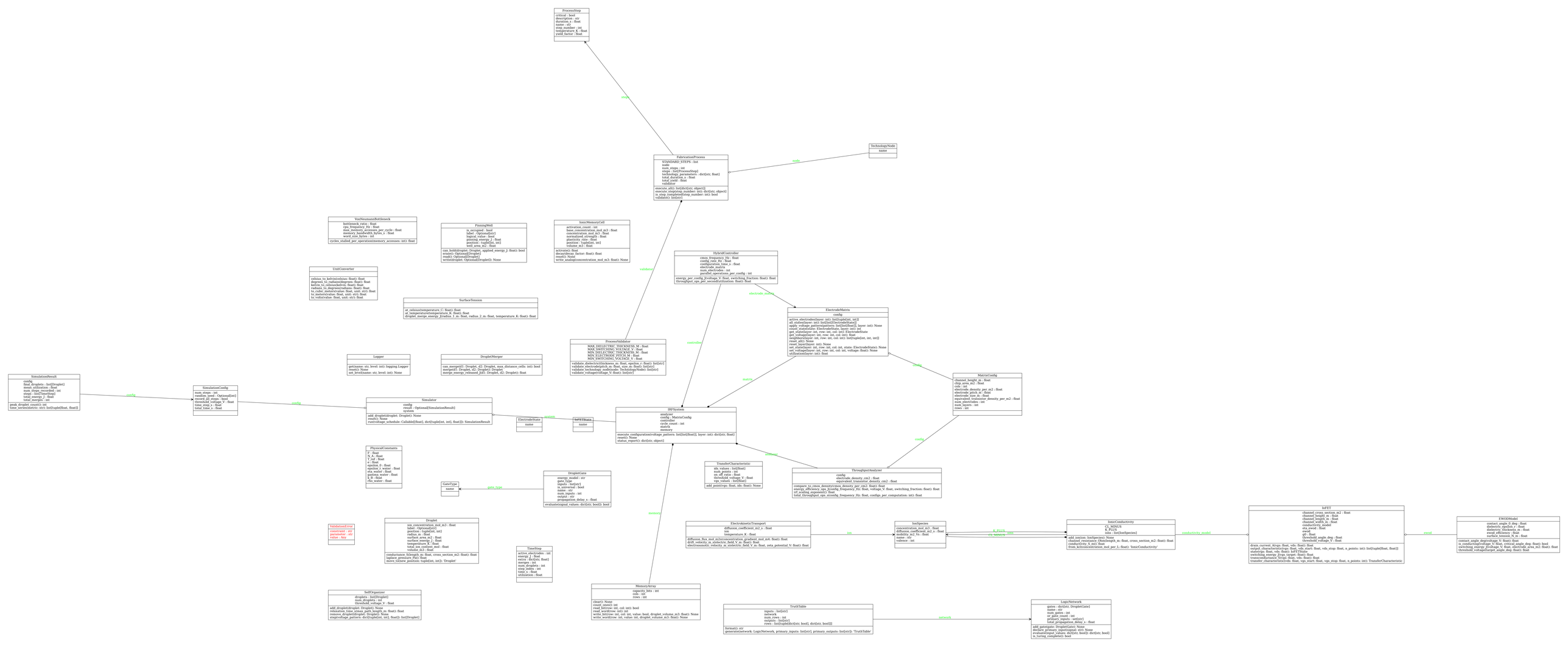

ProcessStep
critical : bool
description : str
duration_s : float
name : str
step_number : int
temperature_K : float
yield_factor : float
steps
FabricationProcess
STANDARD_STEPS : list
node
num_steps : int
steps : list[ProcessStep]
technology_parameters : dict[str, float]
total_duration_s : float
total_yield : float
validator
execute_all(): list[dict[str, object]]
execute_step(step_number: int): dict[str, object]
is_step_completed(step_number: int): bool
validate(): list[str]
node
TechnologyNode
name
VonNeumannBottleneck
bottleneck_ratio : float
cpu_frequency_Hz : float
max_memory_accesses_per_cycle : float
memory_bandwidth_bytes_s : float
word_size_bytes : int
cycles_stalled_per_operation(memory_accesses: int): float
PinningWell
is_occupied : bool
label : Optional[str]
logical_value : bool
pinning_energy_J : float
position : tuple[int, int]
well_area_m2 : float
can_hold(droplet: Droplet, applied_energy_J: float): bool
erase(): Optional[Droplet]
read(): Optional[Droplet]
write(droplet: Optional[Droplet]): None
IonicMemoryCell
activation_count : int
base_concentration_mol_m3 : float
concentration_mol_m3 : float
normalized_strength : float
plasticity_rate : float
position : tuple[int, int]
volume_m3 : float
activate(): float
decay(decay_factor: float): float
reset(): None
write_analog(concentration_mol_m3: float): None
validator
HybridController
cmos_frequency_Hz : float
config_rate_Hz : float
configuration_time_s : float
electrode_matrix
num_electrodes : int
parallel_operations_per_config : int
energy_per_config_J(voltage_V: float, switching_fraction: float): float
throughput_ops_per_second(utilization: float): float
electrode_matrix
UnitConverter
celsius_to_kelvin(celsius: float): float
degrees_to_radians(degrees: float): float
kelvin_to_celsius(kelvin: float): float
radians_to_degrees(radians: float): float
to_cubic_meters(value: float, unit: str): float
to_meters(value: float, unit: str): float
to_volts(value: float, unit: str): float
SurfaceTension
at_celsius(temperature_C: float): float
at_temperature(temperature_K: float): float
droplet_merge_energy_J(radius_1_m: float, radius_2_m: float, temperature_K: float): float
ElectrodeMatrix
config
active_electrodes(layer: int): list[tuple[int, int]]
all_states(layer: int): list[list[ElectrodeState]]
apply_voltage_pattern(pattern: list[list[float]], layer: int): None
count_state(state: ElectrodeState, layer: int): int
get_state(layer: int, row: int, col: int): ElectrodeState
get_voltage(layer: int, row: int, col: int): float
neighbors(layer: int, row: int, col: int): list[tuple[int, int, int]]
reset_all(): None
reset_layer(layer: int): None
set_state(layer: int, row: int, col: int, state: ElectrodeState): None
set_voltage(layer: int, row: int, col: int, voltage: float): None
utilization(layer: int): float
controller
ProcessValidator
MAX_DIELECTRIC_THICKNESS_M : float
MAX_SWITCHING_VOLTAGE_V : float
MIN_DIELECTRIC_THICKNESS_M : float
MIN_ELECTRODE_PITCH_M : float
MIN_SWITCHING_VOLTAGE_V : float
validate_dielectric(thickness_m: float, epsilon_r: float): list[str]
validate_electrode(pitch_m: float, size_m: float): list[str]
validate_technology_node(node: TechnologyNode): list[str]
validate_voltage(voltage_V: float): list[str]
Logger
get(name: str, level: int): logging.Logger
reset(): None
set_level(name: str, level: int): None
DropletMerger
can_merge(d1: Droplet, d2: Droplet, max_distance_cells: int): bool
merge(d1: Droplet, d2: Droplet): Droplet
merge_energy_released_J(d1: Droplet, d2: Droplet): float
config
MatrixConfig
channel_height_m : float
chip_area_m2 : float
cols : int
electrode_density_per_m2 : float
electrode_pitch_m : float
electrode_size_m : float
equivalent_transistor_density_per_m2 : float
num_electrodes : int
num_layers : int
rows : int
matrix
SimulationResult
config
final_droplets : list[Droplet]
mean_utilization : float
num_steps_recorded : int
steps : list[TimeStep]
total_energy_J : float
total_merges : int
peak_droplet_count(): int
time_series(metric: str): list[tuple[float, float]]
config
SimulationConfig
num_steps : int
random_seed : Optional[int]
record_all_steps : bool
threshold_voltage_V : float
time_step_s : float
total_time_s : float
config
Simulator
config
result : Optional[SimulationResult]
system
add_droplet(droplet: Droplet): None
reset(): None
run(voltage_schedule: Callable[[float], dict[tuple[int, int], float]]): SimulationResult
system
ElectrodeState
name
IoFETState
name
IRFSystem
analyzer
config : MatrixConfig
controller
cycle_count : int
matrix
memory
execute_configuration(voltage_pattern: list[list[float]], layer: int): dict[str, float]
reset(): None
status_report(): dict[str, object]
config
analyzer
PhysicalConstants
F : float
N_A : float
T_ref : float
e : float
epsilon_0 : float
epsilon_r_water : float
eta_water : float
gamma_water : float
k_B : float
rho_water : float
TransferCharacteristic
ids_values : list[float]
num_points : int
on_off_ratio : float
threshold_voltage_V : float
vgs_values : list[float]
add_point(vgs: float, ids: float): None
ThroughputAnalyzer
config
electrode_density_cm2 : float
equivalent_transistor_density_cm2 : float
compare_to_cmos_density(cmos_density_per_cm2: float): float
energy_efficiency_ops_J(config_frequency_Hz: float, voltage_V: float, switching_fraction: float): float
irf_scaling_exponent(): float
total_throughput_ops_s(config_frequency_Hz: float, configs_per_computation: int): float
GateType
name
gate_type
DropletGate
energy_model : str
gate_type
inputs : list[str]
is_universal : bool
name : str
num_inputs : int
output : str
propagation_delay_s : float
evaluate(signal_values: dict[str, bool]): bool
memory
IoFET
channel_cross_section_m2 : float
channel_height_m : float
channel_length_m : float
channel_width_m : float
conductivity_model
eta_ewod : float
ewod
g0 : float
threshold_angle_deg : float
threshold_voltage_V : float
drain_current_A(vgs: float, vds: float): float
output_characteristic(vgs: float, vds_start: float, vds_stop: float, n_points: int): list[tuple[float, float]]
state(vgs: float, vds: float): IoFETState
switching_energy_J(vgs_target: float): float
transconductance_S(vgs: float, vds: float): float
transfer_characteristic(vds: float, vgs_start: float, vgs_stop: float, n_points: int): TransferCharacteristic
ewod
EWODModel
contact_angle_0_deg : float
dielectric_epsilon_r : float
dielectric_thickness_m : float
ewod_efficiency : float
surface_tension_N_m : float
contact_angle_deg(voltage_V: float): float
is_conducting(voltage_V: float, critical_angle_deg: float): bool
switching_energy_J(voltage_V: float, electrode_area_m2: float): float
threshold_voltage(target_angle_deg: float): float
ValidationError
constraint : str
parameter : str
value : Any
Droplet
ion_concentration_mol_m3 : float
label : Optional[str]
position : tuple[int, int]
radius_m : float
surface_area_m2 : float
surface_energy_J : float
temperature_K : float
total_ion_content_mol : float
volume_m3 : float
conductance_S(length_m: float, cross_section_m2: float): float
laplace_pressure_Pa(): float
move_to(new_position: tuple[int, int]): 'Droplet'
ElectrokineticTransport
diffusion_coefficient_m2_s : float
ion
temperature_K : float
diffusion_flux_mol_m2s(concentration_gradient_mol_m4: float): float
drift_velocity_m_s(electric_field_V_m: float): float
electroosmotic_velocity_m_s(electric_field_V_m: float, zeta_potential_V: float): float
ion
IonSpecies
concentration_mol_m3 : float
diffusion_coefficient_m2_s : float
mobility_m2_Vs : float
name : str
valence : int
K_PLUS
ions
CL_MINUS
IonicConductivity
CL_MINUS
K_PLUS
ions : list[IonSpecies]
add_ion(ion: IonSpecies): None
channel_resistance_Ohm(length_m: float, cross_section_m2: float): float
conductivity_S_m(): float
from_kcl(concentration_mol_per_L: float): 'IonicConductivity'
conductivity_model
TimeStep
active_electrodes : int
energy_J : float
extra : dict[str, float]
merges : int
num_droplets : int
step_index : int
time_s : float
utilization : float
SelfOrganizer
droplets : list[Droplet]
num_droplets : int
threshold_voltage_V : float
add_droplet(droplet: Droplet): None
relaxation_time_s(max_path_length_m: float): float
remove_droplet(droplet: Droplet): None
step(voltage_pattern: dict[tuple[int, int], float]): list[Droplet]
MemoryArray
capacity_bits : int
cols : int
rows : int
clear(): None
count_ones(): int
read_bit(row: int, col: int): bool
read_word(row: int): int
write_bit(row: int, col: int, value: bool, droplet_volume_m3: float): None
write_word(row: int, value: int, droplet_volume_m3: float): None
TruthTable
inputs : list[str]
network
num_rows : int
outputs : list[str]
rows : list[tuple[dict[str, bool], dict[str, bool]]]
format(): str
generate(network: LogicNetwork, primary_inputs: list[str], primary_outputs: list[str]): 'TruthTable'
network
LogicNetwork
gates : dict[str, DropletGate]
name : str
num_gates : int
or_gate_count : int
primary_inputs : set[str]
total_propagation_delay_s : float
add_gate(gate: DropletGate): None
declare_primary_input(signal: str): None
evaluate(input_values: dict[str, bool]): dict[str, bool]
is_turing_complete(): bool